

Dataset Used: Gold_Price Dataset (for classification experiments)

EXPERIMENT 1

```
In [35]: # Basic Python Data Structures
#List
a = [1, 2, 3]
a.append(4)
print("List:", a)

#Tuple
t = (10, 20, 30)
print("Tuple:", t)

#Set
s = {1, 2, 2, 3}
print("Set:", s)

#Dictionary
d = {"name": "musharraf", "age": 23}
print("Dictionary:", d)
```

```
List: [1, 2, 3, 4]
Tuple: (10, 20, 30)
Set: {1, 2, 3}
Dictionary: {'name': 'musharraf', 'age': 23}
```

Gold_price_2025 NewDataset LOADING

Model ready

```
In [36]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# Load dataset
df = pd.read_csv("Gold_price_2025 new.csv")
# Inspect data
print(df.head())
print(df.info())
# Handle missing values
df = df.dropna()
# Remove duplicates
df = df.drop_duplicates()
# Convert columns to float
price_columns = ['priceOpen', 'priceHigh', 'priceLow', 'priceClose']
for col in price_columns:
    df[col] = df[col].astype(str).str.replace(',', '.').astype(float)
df['volume'] = df['volume'].astype(str).str.replace(',', '.').astype(float)
```

```
# Feature and target
X = df[['priceOpen', 'priceHigh', 'priceLow']]
y = df['priceClose']
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

	timeOpen	timeClose	timeHigh	timeLow	priceOpen \
0	1753531200000	1753617599999	1753589280000	1753534320000	0,006715796828
1	1753444800000	1753531199999	1753499400000	1753451940000	0,006643911641
2	1753358400000	1753444799999	1753358880000	1753415760000	0,006750933166
3	1753272000000	1753358399999	1753312560000	1753350120000	0,007042174606
4	1753185600000	1753271999999	1753253520000	1753265040000	0,007057862602

	priceHigh	priceLow	priceClose	volume
0	0,006865782367	0,006714274853	0,006842611523	0
1	0,006718620251	0,006642475657	0,006715796828	0
2	0,006752793583	0,006642331031	0,006643911641	0
3	0,007053935013	0,006749428149	0,006750933166	0
4	0,007093906453	0,006988306188	0,007042174606	130,07

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 664 entries, 0 to 663

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	timeOpen	664 non-null	int64
1	timeClose	664 non-null	int64
2	timeHigh	664 non-null	int64
3	timeLow	664 non-null	int64
4	priceOpen	664 non-null	object
5	priceHigh	664 non-null	object
6	priceLow	664 non-null	object
7	priceClose	664 non-null	object
8	volume	664 non-null	object

dtypes: int64(4), object(5)

memory usage: 46.8+ KB

None

EXPERIMENT 2 Euclidean Distance

```
In [37]: def euclidean_distance(p1, p2):
          return np.sqrt(np.sum((p1 - p2)**2))

p1 = np.array([2, 3])
p2 = np.array([5, 7])

print("Euclidean Distance:", euclidean_distance(p1, p2))
```

Euclidean Distance: 5.0

EXPERIMENT 3 K-Nearest Neighbors

```
In [38]: from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Create model
knn = KNeighborsRegressor(n_neighbors=3)

# Train
knn.fit(X_train, y_train)

# Predict
pred_knn = knn.predict(X_test)

# Evaluation
print("MSE:", mean_squared_error(y_test, pred_knn))
print("R2 Score:", r2_score(y_test, pred_knn))
```

MSE: 2.7781685531655172e-05

R2 Score: 0.971675144510534

EXPERIMENT 4 Decision Tree

```
In [39]: from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Create the Decision Tree Regressor
dt = DecisionTreeRegressor(random_state=42)

# Train the model
dt.fit(X_train, y_train)

# Predict on test set
pred_dt = dt.predict(X_test)

# Evaluate the model
print("Mean Squared Error (MSE):", mean_squared_error(y_test, pred_dt))
print("R2 Score:", r2_score(y_test, pred_dt))
```

Mean Squared Error (MSE): 3.372116231353893e-05

R2 Score: 0.9656195428322916

EXPERIMENT 5 Classification Metrics

```
In [40]: from sklearn.metrics import mean_squared_error
import numpy as np

mse = mean_squared_error(y_test, pred_dt)
rmse = np.sqrt(mse)
```

```
print("MSE:", mse)
print("RMSE:", rmse)
```

MSE: 3.372116231353893e-05

RMSE: 0.005806992536032652

In [41]: `from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score`
`import numpy as np`

```
mae = mean_absolute_error(y_test, pred_dt)
mse = mean_squared_error(y_test, pred_dt)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, pred_dt)
```

```
print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

MAE: 0.0015913649125187973

MSE: 3.372116231353893e-05

RMSE: 0.005806992536032652

R2 Score: 0.9656195428322916

EXPERIMENT 6 K-Means Clustering

In [42]: `from sklearn.cluster import KMeans`

```
# Fit K-Means with 3 clusters
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X)

# Print cluster centers
print("Cluster Centers:\n", kmeans.cluster_centers_)
```

Cluster Centers:

```
[[0.01526677 0.01561772 0.01493794]
 [0.13412633 0.14490044 0.11935016]
 [0.05212854 0.06007844 0.04900581]]
```

C:\Users\Lenovo\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=3.

```
warnings.warn(
```

EXPERIMENT 7 Decision Tree with Entropy

In [43]: `from sklearn.tree import DecisionTreeRegressor`
`from sklearn.metrics import mean_squared_error, r2_score`

```
# Create Decision Tree Regressor
dt_reg = DecisionTreeRegressor(criterion="squared_error") # or "friedman_mse"
dt_reg.fit(X_train, y_train)
```

```

# Predict
pred = dt_reg.predict(X_test)

# Evaluate
mse = mean_squared_error(y_test, pred)
r2 = r2_score(y_test, pred)

print("MSE:", mse)
print("R2 Score:", r2)

```

MSE: 3.410698972223908e-05

R2 Score: 0.965226171969935

EXPERIMENT 8 Random Forest

```

In [44]: from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Create Random Forest Regressor
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

# Predict
pred_rf = rf.predict(X_test)

# Evaluate
mse = mean_squared_error(y_test, pred_rf)
r2 = r2_score(y_test, pred_rf)

print("Random Forest MSE:", mse)
print("Random Forest R2 Score:", r2)

```

Random Forest MSE: 2.68014548462464e-05

Random Forest R2 Score: 0.9726745400468094

EXPERIMENT 9 Naive Bayes

```

In [45]: import pandas as pd
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

# Example: categorize priceClose into 3 levels: low, medium, high
df['PriceCategory'] = pd.qcut(df['priceClose'], q=3, labels=[0,1,2])

X = df[['priceOpen', 'priceHigh', 'priceLow']] # features
y = df['PriceCategory']                       # target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

# Gaussian Naive Bayes
nb = GaussianNB()

```

```
nb.fit(X_train, y_train)

pred_nb = nb.predict(X_test)
print("Naive Bayes Accuracy:", accuracy_score(y_test, pred_nb))
```

Naive Bayes Accuracy: 0.9548872180451128

EXPERIMENT 10 Logistic Regression

```
In [46]: from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Create Linear Regression model
lr = LinearRegression()
lr.fit(X_train, y_train)

# Predict
pred_lr = lr.predict(X_test)

# Evaluate
mse = mean_squared_error(y_test, pred_lr)
r2 = r2_score(y_test, pred_lr)

print("Linear Regression MSE:", mse)
print("Linear Regression R2 Score:", r2)
```

Linear Regression MSE: 0.32068679943560346

Linear Regression R2 Score: 0.5221037240761257

EXPERIMENT 11: Naive Bayes Classifier

```
In [47]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Load YOUR dataset (correct file name)
df = pd.read_csv("Gold_price_2025 new.csv")

# Convert numeric columns
price_columns = ['priceOpen', 'priceHigh', 'priceLow', 'priceClose']

for col in price_columns:
    df[col] = df[col].astype(str).str.replace(',', '.').astype(float)

# Create classification target (IMPORTANT)
df['target'] = (df['priceClose'] > df['priceClose'].mean()).astype(int)

# Features and target
X = df[['priceOpen', 'priceHigh', 'priceLow']]
y = df['target']

# Train-test split
```

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Model
model = GaussianNB()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

```

Accuracy: 0.9924812030075187

Experiment 12: Simple Linear Regression using sklearn

```

In [48]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
df = pd.read_csv("Gold_price_2025 new.csv")
# Convert numeric columns
cols = ['priceOpen', 'priceClose']
for col in cols:
    df[col] = df[col].astype(str).str.replace(',', '.').astype(float)
# Feature and target (Simple Linear Regression = one feature)
X = df[['priceOpen']]
y = df[['priceClose']]
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Model
model = LinearRegression()
model.fit(X_train, y_train)
# Prediction
y_pred = model.predict(X_test)
# Evaluation
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

```

MSE: 3.6434332198960374e-05

R2 Score: 0.9628533267639624

Experiment 13: Regression using Ordinary Least Squares (OLS)

```
In [49]: import pandas as pd
import statsmodels.api as sm

# Load dataset
df = pd.read_csv("Gold_price_2025 new.csv")
# Convert numeric columns
cols = ['priceOpen', 'priceClose']
for col in cols:
    df[col] = df[col].astype(str).str.replace(',', '.').astype(float)
# Feature and target
X = df[['priceOpen']]
y = df['priceClose']
# Add constant (IMPORTANT for intercept)
X = sm.add_constant(X)
# Build OLS model
model = sm.OLS(y, X).fit()
# Print summary
print(model.summary())
```

```

                                OLS Regression Results
=====
Dep. Variable:                  priceClose    R-squared:                 0.955
Model:                            OLS       Adj. R-squared:            0.955
Method:                 Least Squares      F-statistic:                 1.399e+04
Date:                  Wed, 15 Apr 2026     Prob (F-statistic):          0.00
Time:                      04:29:13        Log-Likelihood:             2445.5
No. Observations:          664             AIC:                     -4887.
Df Residuals:              662             BIC:                     -4878.
Df Model:                   1
Covariance Type:           nonrobust
=====
               coef      std err          t      P>|t|      [0.025      0.975]
-----
const          0.0008      0.000      2.228      0.026      9.03e-05      0.001
priceOpen      0.9696      0.008    118.295      0.000      0.953      0.986
=====
Omnibus:                 454.620    Durbin-Watson:           2.401
Prob(Omnibus):            0.000    Jarque-Bera (JB):        36920.034
Skew:                    2.257    Prob(JB):                0.00
Kurtosis:                39.250    Cond. No.               34.7
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Experiment 14: Locally Weighted Regression (LWR)

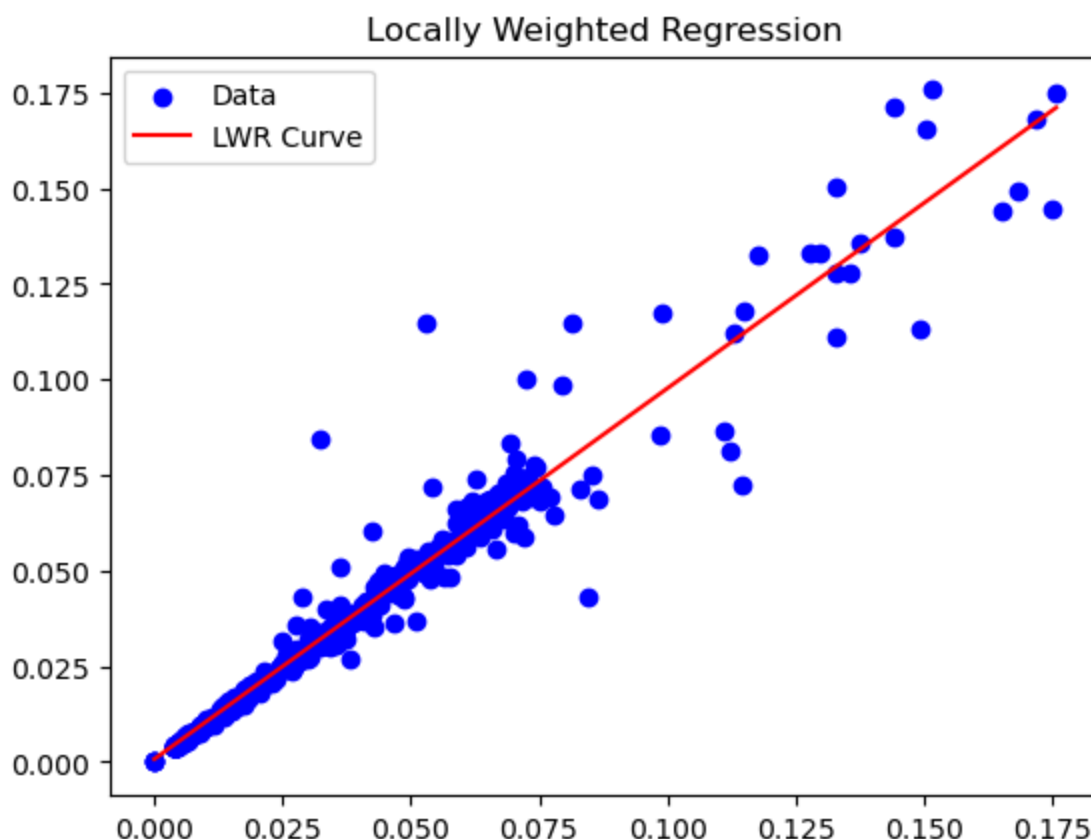
```
In [50]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
# Load dataset
```



```

df = pd.read_csv("Gold_price_2025 new.csv")
# Convert numeric columns
cols = ['priceOpen', 'priceClose']
for col in cols:
    df[col] = df[col].astype(str).str.replace(',', '.').astype(float)
# Feature and target
X = df['priceOpen'].values
y = df['priceClose'].values
# Locally Weighted Regression function
def lwr(x_query, X, y, tau=0.5):
    m = len(X)
    weights = np.exp(-(X - x_query)**2 / (2 * tau**2))
    W = np.diag(weights)
    X_ = np.column_stack((np.ones(m), X))
    theta = np.linalg.pinv(X_.T @ W @ X_) @ (X_.T @ W @ y)
    return np.array([1, x_query]) @ theta
# Predictions
X_test = np.linspace(min(X), max(X), 100)
y_pred = [lwr(x, X, y, tau=0.5) for x in X_test]
# Plot
plt.scatter(X, y, color='blue', label='Data')
plt.plot(X_test, y_pred, color='red', label='LWR Curve')
plt.legend()
plt.title("Locally Weighted Regression")
plt.show()

```



Experiment 15: Multi-class SVM (Support Vector Machine)

```
In [51]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
# Load dataset
df = pd.read_csv("Gold_price_2025 new.csv")
# Convert numeric columns
cols = ['priceOpen', 'priceHigh', 'priceLow', 'priceClose']
for col in cols:
    df[col] = df[col].astype(str).str.replace(',', '.').astype(float)
# Create multi-class target (VERY IMPORTANT)
# Example: classify priceClose into 3 classes
df['target'] = pd.qcut(df['priceClose'], q=3, labels=[0, 1, 2])
# Features and target
X = df[['priceOpen', 'priceHigh', 'priceLow']]
y = df['target']
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# SVM Model (One-vs-One by default for multi-class)
model = SVC(kernel='linear')
model.fit(X_train, y_train)
# Prediction
y_pred = model.predict(X_test)
# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Accuracy: 0.39849624060150374

Classification Report:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	50
1	0.35	1.00	0.52	43
2	1.00	0.25	0.40	40
accuracy			0.40	133
macro avg	0.45	0.42	0.31	133
weighted avg	0.41	0.40	0.29	133

```
C:\Users\Lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1833:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
C:\Users\Lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1833:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
C:\Users\Lenovo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1833:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

Experiment 16: Case Study (Prediction + Classification)

```
In [52]: from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
# Features and target
X = df[['priceOpen', 'priceHigh', 'priceLow']]
y = df['priceClose']
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Model
reg = LinearRegression()
reg.fit(X_train, y_train)
# Prediction
y_pred = reg.predict(X_test)
print("R2 Score:", r2_score(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
```

R2 Score: 0.9768451464226615

MSE: 2.2710825863043724e-05

Experiment 17: CNN (Convolutional Neural Network) using TensorFlow/Keras

```
In [53]: import tensorflow as tf
from tensorflow.keras import layers, models

# Build CNN model
model = models.Sequential()

# Convolution Layer 1
model.add(layers.Conv2D(32, (3,3), activation='relu', input_shape=(64, 64, 3)))
model.add(layers.MaxPooling2D((2,2)))

# Convolution Layer 2
model.add(layers.Conv2D(64, (3,3), activation='relu'))
model.add(layers.MaxPooling2D((2,2)))

# Flatten layer
model.add(layers.Flatten())

# Fully connected layer
model.add(layers.Dense(64, activation='relu'))

# Output layer (example: 3 classes)
model.add(layers.Dense(3, activation='softmax'))
```

```
# Compile model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()
```

C:\Users\Lenovo\anaconda3\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_1"

Layer (type)	Output Shape	
conv2d_2 (Conv2D)	(None, 62, 62, 32)	
max_pooling2d_2 (MaxPooling2D)	(None, 31, 31, 32)	
conv2d_3 (Conv2D)	(None, 29, 29, 64)	
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 64)	
flatten_1 (Flatten)	(None, 12544)	
dense_2 (Dense)	(None, 64)	
dense_3 (Dense)	(None, 3)	

Total params: 822,467 (3.14 MB)

Trainable params: 822,467 (3.14 MB)

Non-trainable params: 0 (0.00 B)

Experiment 18: Grid Search and Random Search using Logistic Regression

```
In [54]: import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
# Convert numeric columns
cols = ['priceOpen', 'priceHigh', 'priceLow', 'priceClose']
for col in cols:
    df[col] = df[col].astype(str).str.replace(',', '.').astype(float)
# Create classification target
df['target'] = (df['priceClose'] > df['priceClose'].mean()).astype(int)
X = df[['priceOpen', 'priceHigh', 'priceLow']]
y = df['target']
# Split data
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Model
model = LogisticRegression()
# Hyperparameter grid
param_grid = {
    'C': [0.01, 0.1, 1, 10],
    'solver': ['liblinear']
}
# Grid Search
grid = GridSearchCV(model, param_grid, cv=5)
grid.fit(X_train, y_train)
# Best model
best_model = grid.best_estimator_
# Prediction
y_pred = best_model.predict(X_test)
print("Best Parameters:", grid.best_params_)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Best Parameters: {'C': 10, 'solver': 'liblinear'}
Accuracy: 0.924812030075188

In []: